

Session report: NGS sampling depletion, and the F/J extreme-variant investigation

August 10, 2026

Abstract

This session covered two mostly independent threads. Part 1 fixes a physical-realism bug in the directed-evolution simulator (`sequence_classesV1.py`): variants sampled for NGS were never removed from the pool that continues on to the next pipeline step, which silently understated the impact of the sampling/dilution factor. Part 2 investigates why, once we look at the top-50,000 (rather than top-5,000) variants by true fitness, an MLP trained to recover the fitness landscape from NGS-derived data starts missing an entire block of them – traces this to the pairwise coupling term J dominating the score of extreme-tail variants, attempts an architectural fix (an explicit bilinear/Potts head), finds it only marginally helps, and diagnoses why. Part 3 steps back and asks the more fundamental question: is J 's outsized role in the extreme tail a genuine, expected property of the landscape, or an artifact of how F and J were estimated in the first place.

Contents

1	NGS sampling should deplete the pool it measures	2
1.1	The problem	2
1.2	Fix 1: deplete the pool at each NGS checkpoint	2
1.3	Fix 2: Poisson sampling does not saturate at <code>dilution_factor=1</code>	2
1.4	Verification	3
2	The extreme-tail investigation: does the MLP recover the best variants?	3
2.1	Starting observation	3
2.2	Hypothesis: F vs. J	3
2.3	Verification	3
2.4	Attempted fix: an explicit bilinear (Potts) head	4
2.5	Result: barely any improvement	4
2.6	Diagnosing why	4
2.7	Leading hypothesis	5
3	Why does J dominate the extreme tail – and is our J trustworthy?	5
3.1	Why extreme-tail dominance by J is, at least partly, expected	6
3.2	Candidate sources of genuine bias / overestimation in this specific pipeline	6
3.3	How this could be checked (not done today)	7
3.4	Tentative conclusion	7
4	Summary of files touched today	7

1 NGS sampling should deplete the pool it measures

1.1 The problem

In the real protocol, pulling an aliquot for NGS (pipetting, PCR amplification, sequencing) consumes those molecules – they cannot also continue on to the next step of the pipeline. In `Protocol.loop_DE()`, however, `NGS()` sampled a copy of the pool into a temporary `Lambda` pipeline and returned only the read counts; `self.lambda0`, `self.lambda2`, and `self.lambda3` were left completely untouched. The same variants that were “consumed” by an NGS checkpoint kept flowing downstream at full strength, as if the checkpoint had no cost. This silently *understated* the importance of the `dilution_factor` parameter: no matter how small the dilution factor (i.e. how large a fraction of the pool is pipetted out), the downstream pool size never actually dropped.

1.2 Fix 1: deplete the pool at each NGS checkpoint

A new method, `Protocol.ngs_and_deplete()`, was added: it performs the same pipetting $\rightarrow$ PCR $\rightarrow$ sequencing pipeline as the existing `NGS()`, but additionally returns the source pool with the pipetted aliquot subtracted off (floored at 0). `loop_DE()` now calls this at all three checkpoints, with $g := \text{ngs_and_deplete}$:

$$(\lambda'_0, \lambda_0) \leftarrow g(\lambda_0), \quad (\lambda'_2, \lambda_2) \leftarrow g(\lambda_2), \quad (\lambda'_3, \lambda_3) \leftarrow g(\lambda_3)$$

so that `initial_sampling()`, `selectivity()`, and `bacterial_amplification()` all operate on the *depleted* pool. The public `NGS()` method itself was kept non-destructive (it now just calls `_ngs_and_deplete` and discards the depleted copy), since it is also used standalone elsewhere for what-if measurements on a frozen snapshot (e.g. `analysisV1.plot_lambda3p_dilution_scatter`, which re-samples the same `lambda3` at several dilution factors and must not mutate it between draws).

1.3 Fix 2: Poisson sampling does not saturate at `dilution_factor=1`

Testing the fix at `dilution_factor=1` (all of the pool nominally pipetted out) revealed a second, deeper bug: many variants still survived depletion. `Lambda.sample_sequences()` drew the aliquot as

$$X_s \sim \text{Poisson}\left(\text{pool}_s \cdot \frac{1}{\text{dilution_factor}}\right),$$

which is a good approximation of “sample a small fraction without replacement” only when the fraction $p = 1/\text{dilution_factor}$ is small. As $p \rightarrow 1$, Poisson’s own variance ($\text{sd} \approx \sqrt{\text{mean}}$) means the draw lands *below* the true count on roughly half of all variants purely by chance, even though its mean exactly equals the pool size – leaving a nonzero, unconsumed residual for about half the variants, *even at `dilution_factor=1`*.

The fix replaces this with

$$X_s \sim \text{Binomial}(\text{pool}_s, p),$$

i.e. each molecule independently has probability p of landing in the aliquot. This is exact rather than an approximation: it guarantees $X_s \leq \text{pool}_s$ always, and at $p = 1$ every molecule is included with certainty (deterministic full consumption).

1.4 Verification

	dilution_factor=1	dilution_factor=10
pool sum before depletion	1,000,000	1,000,000
pool sum after depletion	0	900,117
variants surviving (of 200 tested)	0	—

At `dilution_factor=1`, the pool is now fully and exactly consumed (0 survivors), matching the physical expectation. At `dilution_factor=10`, roughly 90% of the pool survives, as expected. A full `loop_DE()` round at `dilution_factor=1` also runs end-to-end without error, propagating `lambda0=0` into the rest of the pipeline correctly (a legitimate, if degenerate, consequence of spending 100% of the input library on QC).

Files changed: `Modelization_V1/lib/sequence_classesV1.py` (`Lambda.sample_sequences`, `Protocol.NGS`, new `Protocol._ngs_and_deplete`, `Protocol.loop_DE`).

2 The extreme-tail investigation: does the MLP recover the best variants?

2.1 Starting observation

In `MLP_for_anticorrelated_weights.ipynb` (real AAV9-derived $F_{\text{viab}}/J_{\text{viab}}$, anticorrelated $F_{\text{sel}}/J_{\text{sel}}$ with target correlation -0.2), a `ProfileMLP` is trained to predict NGS log-enrichment from sequence, then its recovery of the TRUE top- K global-optimum variants (found by exhaustively scoring all 20^7 possible sequences) is checked. At $K = 5,000$, almost all of these land where they should in the MLP’s predicted distribution, except for one visibly mispredicted block. Raising K to 50,000 revealed a second, distinct block – the mispredicted variants no longer looked like scattered outliers around a single well-predicted cluster, but like two genuinely separate populations.

2.2 Hypothesis: F vs. J

The ground-truth score decomposes exactly as

$$\text{score}(s) = \underbrace{\sum_i F[s_i, i]}_{F\text{-part (additive)}} + \underbrace{\sum_{i < j} J[i, j, s_i, s_j]}_{J\text{-part (pairwise)}},$$

and the working hypothesis was that the well-predicted block is dominated by F (a simple, low-dimensional, easy-to-learn additive term) while the mispredicted block is dominated by J (a much higher-dimensional pairwise term that a generic MLP has to approximate implicitly through hidden-layer nonlinearity, rather than being given the structure directly).

2.3 Verification

New cells were added to `MLP_for_anticorrelated_weights.ipynb` decomposing each of the TRUE top-50,000 variants’ scores into F -part/ J -part (via `compute_F_only(seq, F) = total_score(seq) - F_part`), and comparing:

- the variance share of F -part vs. J -part within the top-50,000 group, against the same split on a random 2,000,000-variant sample;

- a per-variant z-scored comparison (F_z , J_z , standardized against the random sample’s own mean/std) showing what fraction of top- K variants have $J_z > F_z$.

Both confirmed the hypothesis directionally: J ’s share of the score variance is much larger among the extreme top-50,000 than in a random sample. (One caveat, discovered only while writing this report: the printed variance-share percentages for the top- K group came out well above 100% (e.g. “ J explains 5030.0%”). This is a real but misleading artifact of $\text{Var}(\text{total}) = \text{Var}(F) + \text{Var}(J) + 2\text{Cov}(F, J)$: within a *selected* top- K set, F -part and J -part are strongly *negatively* covariant – a variant needs a large J to compensate for a mediocre F to make the cut, or vice versa – so $2\text{Cov}(F, J)$ is large and negative, shrinking the total variance in the denominator far below either individual term. The z-score/scatter comparison is not affected by this and remains the more trustworthy diagnostic.)

2.4 Attempted fix: an explicit bilinear (Potts) head

Since $x_i^\top W_{ij} x_j = W_{ij}[s_i, s_j]$ exactly when x is one-hot, a **BilinearHead** module was added computing $\sum_{i < j} x_i^\top W_{ij} x_j$ – the same indicator-feature form as the true $J[i, j, a, b]$ – and summed residually with the existing **ProfileMLP** branch (**ProfileMLPBilinear**). The idea: give the model direct, unmediated access to the pairwise structure instead of making it approximate that structure implicitly. This was implemented and trained in a new notebook, `MLP_bilinear_head_anticorrelated.ipynb`, training a baseline plain **ProfileMLP** and a **ProfileMLP**+bilinear-head model side by side on the identical train/val split.

2.5 Result: barely any improvement

Held-out test set	viability r	selectivity r
baseline (plain MLP)	0.9651	0.9021
+ bilinear head	0.9652	0.9095

TRUE top-50,000 set	$r(\text{resid}, F\text{-part})$	$r(\text{resid}, J\text{-part})$	mean resid
viability, baseline	0.9767	−0.9911	7.457
viability, + bilinear	0.9731	−0.9867	6.943
selectivity, baseline	0.4890	−0.6418	6.095
selectivity, + bilinear	0.4396	−0.5462	5.398

(residual := predicted – true, on the TRUE top-50,000 variants)

Test-set correlation is essentially unchanged, and $r(\text{resid}, J\text{-part})$ – which should shrink toward 0 if the bilinear head were correctly absorbing the pairwise structure – barely moved for viability and only modestly improved for selectivity. Giving the model the “right” functional form was not, by itself, enough.

2.6 Diagnosing why

Three candidate explanations were tested.

(a) Rare training-data coverage of the relevant pairs – refuted. For each TRUE top-50,000 variant, its dominant (largest- $|J|$) contributing pair (i, j, a, b) was looked up in the 20,000-sequence training set. Median training-set count of that dominant pair: **52**, essentially identical to a random training-distribution sequence’s dominant pair (median **51**). Since sequences are drawn

i.i.d. uniformly, every (i, j, a, b) combination is represented with equal expected frequency regardless of its J -magnitude – so the extreme variants do not rely on unusually rare training examples.

(b) Explicit regularization / early stopping – refuted. The bilinear model was retrained with dropout and weight decay off and early stopping disabled (patience = 1000, epochs = 1000), driving training MSE down to 0.0002 (essentially exact memorization of all 8,500 training rows).

config	train MSE	test r	$r(\text{resid}, F)$	$r(\text{resid}, J)$
regularized (dropout 0.1, patience 20)	0.0644	0.9652	0.9731	-0.9867
unregularized (dropout 0, patience 1000)	0.0002	0.9566	0.9691	-0.9817

Despite near-perfect training-set memorization, $r(\text{resid}, J\text{-part})$ barely moved ($-0.987 \rightarrow -0.982$), and held-out test r got slightly *worse*. Explicit regularization is not the driver of the bias.

(c) More training data – inconclusive (confounded). A retrain using the cached 2,000,000-sequence dataset (vs. 20,000) gave val MSE ≈ 3.85 , test $r = 0.31$, $r(\text{resid}, J\text{-part}) = -0.82$ – worse-looking numbers, but **not a clean comparison**: the total sequencing budget D is fixed, so spreading it over $100\times$ more distinct variants means $\sim 100\times$ fewer reads per variant, and hence intrinsically noisier NGS-derived labels, independent of any modeling choice. Training hyperparameters (batch size, epoch budget, patience) also differed between the two runs. This experiment would need to be re-run with D scaled up proportionally to diversity (holding per-variant read depth constant) to be a fair test of “more data” in isolation.

2.7 Leading hypothesis

The pattern that survives both refutations – residual systematically *underestimating* variants with large true J -part, essentially unchanged whether or not explicit regularization is present – is consistent with **implicit minimum-norm shrinkage**: with $\approx 8,540$ combined F/J parameters against $\approx 8,500$ effective training rows (train split of the 20,000-sequence dataset), the regression problem is only barely determined. Gradient descent on an over/critically-parameterized model is known to converge toward the minimum-norm solution among all those that fit the training data equally well – an implicit shrinkage-toward-zero that has nothing to do with dropout or weight decay, and that specifically underestimates the largest true parameter values. This was not independently confirmed today (it would need, e.g., a properly-controlled large- N retrain, or an explicit low-rank / shared-strength reparameterization of the bilinear head) and should be treated as the leading hypothesis, not a settled conclusion.

Files added/changed: `Modelization_V1/notebooks/selectivity_weight_regimes/MLP_for_anticorrelated_weights.ipynb` (new F/J -decomposition cells); `Modelization_V1/notebooks/selectivity_weight_regimes/MLP_bilinear_head_anticorrelated.ipynb` (new notebook, baseline vs. bilinear-head comparison).

3 Why does J dominate the extreme tail – and is our J trustworthy?

The user’s question going into this section: is it *normal* that a handful of pairwise couplings end up mattering more than the entire additive profile for the best variants, or is this symptomatic of our particular estimate of J being biased upward?

3.1 Why extreme-tail dominance by J is, at least partly, expected

Term-count amplification. For $L = 7$ positions there are only 7 additive terms but $\binom{7}{2} = 21$ pairwise terms – three times as many. If individual F and J entries are of roughly comparable typical magnitude (plausible here, since these are real AAV9-recovered weights, not the synthetically down-scaled ones used elsewhere in this codebase), then for a random sequence

$$\text{Var}(J\text{-part}) \approx 21 \cdot \text{Var}(J_{\text{entry}}) > \text{Var}(F\text{-part}) \approx 7 \cdot \text{Var}(F_{\text{entry}})$$

purely from having three times as many summands, with no need for any single J entry to be unusually large. This alone is a real, expected structural effect – not a bias.

Order-statistics amplification. The score of a random sequence is (approximately, for largely independent terms) a sum of two roughly-Gaussian components, F -part and J -part. The extreme tail of a sum of independent random variables is disproportionately driven by whichever component has the larger variance – a component with $3\times$ the variance contributes correspondingly more to *how* a sum ends up in its own extreme tail. So even before considering estimation issues, J should be expected to explain a *larger* share of what makes the top-50,000 variants extreme than it explains of the population as a whole – qualitatively exactly what was measured.

Biological plausibility. This also matches basic protein biophysics: there is exactly one profile that is additively optimal (best amino acid at every position, independent of context), but combinatorially many ways for a sequence to be exceptional via coordinated, context-dependent interactions (structural contacts, compensatory pairs, packing interactions). It would be somewhat surprising if the *most* exceptional variants in a real capsid landscape were reachable purely through independent, single-residue effects.

3.2 Candidate sources of genuine bias / overestimation in this specific pipeline

The above arguments say J -dominance of the tail is expected in kind, but they do not say the *measured magnitude* of that dominance is trustworthy. There are concrete reasons to suspect our particular J_{viab} (and, by construction, $J_{\text{sel,anti}}$) may be inflated:

(i) **Estimation noise inflation in the recovery procedure.** $F_{\text{viab}}/J_{\text{viab}}$ are not measured directly; they are *recovered* from a ProfileMLP that was itself fitted to real AAV9 data, via `extract_effective_F/extract_effective_J` (AAV9_profile_model.ipynb), which probe the fitted MLP with single- and double-mutant scans over $M = 256$ background sequences. Critically, $\hat{J}_{ij}(a, b)$ is a *four-term* contrast,

$$\hat{J}_{ij}(a, b) = \langle \text{double}(a, b) - \text{single}_i(a) - \text{single}_j(b) + \text{base} \rangle_{\text{backgrounds}},$$

against $\hat{F}_i(a)$'s single-term contrast $\langle \text{single}_i(a) - \text{base} \rangle$. Each extra subtracted term adds its own estimation noise, and *noise inflates apparent variance* ($\text{Var}(\text{estimate}) = \text{Var}(\text{truth}) + \text{Var}(\text{noise})$, never less). With $\sim 60\times$ more free entries to estimate (8,400 vs. 140) from the same 256 backgrounds, and a noisier per-entry contrast, $\hat{J}$ is the more likely of the two to carry a meaningfully inflated apparent variance relative to its true value.

(ii) **Aliasing of higher-order epistasis onto the pairwise term.** The recovery procedure *assumes* the underlying landscape is exactly additive + pairwise. Nothing guarantees the real AAV9 MLP’s fitted landscape is free of third-order-and-above interactions; if it is not, that residual structure has nowhere else to go in a Potts-shaped extraction and gets folded into $\hat{J}$, inflating its apparent importance for reasons that have nothing to do with true pairwise epistasis.

(iii) **Propagation into the selectivity landscape.** $J_{\text{sel,anti}}$ is not estimated independently – it is built by `initialize_anticorrelated_weights`, which mixes J_{viab} ’s *own* values and scale with fresh noise ($\text{Var}(J_{\text{sel}}) = \text{Var}(J_{\text{viab}})$ by construction, at any correlation level). Whatever inflation is present in J_{viab} is therefore inherited by $J_{\text{sel,anti}}$ automatically, so this is not a concern isolated to the viability landscape.

3.3 How this could be checked (not done today)

- **Bootstrap stability of $\hat{J}$.** Re-run `extract_effective_J` on several disjoint subsets of the 256 backgrounds and compare entry-by-entry variance across subsets to the entries’ own typical magnitude. If $\hat{J}$ entries are unstable (bootstrap std comparable to or larger than the entry values themselves), that is direct evidence of estimation-noise inflation rather than genuine signal.
- **Unconditional $\text{Var}(F)$ vs. $\text{Var}(J)$, entrywise.** Compare the raw entrywise variance of F_{viab} against J_{viab} directly (not conditioned on the top- K selection), to separate the “3× as many terms” structural effect (Section 4.1) from a genuine per-entry magnitude difference.
- **Compare against a synthetic ground truth.** Re-run the same top- K F/J-dominance analysis on `initialize_random_weights`’ synthetic F/J (where J is deliberately scaled down by $\frac{1}{10}$ relative to F , and is not subject to any recovery-noise inflation since it is exact by construction). If J -dominance of the extreme tail *disappears* there, that strengthens the case that today’s result is at least partly an artifact of the AAV9 recovery pipeline rather than a universal property of Potts-shaped landscapes.

3.4 Tentative conclusion

Both effects are almost certainly present simultaneously: term-count and order-statistics amplification (Section 4.1) are structurally guaranteed to make J matter disproportionately in the extreme tail, in any Potts-shaped landscape, real or synthetic. Whether the specific *magnitude* we observed today is entirely genuine, or partly inflated by recovery-procedure noise and higher-order-epistasis aliasing (Section 4.2), was not resolved this session and is the most valuable open question to settle before drawing further conclusions from J -vs- F comparisons on this particular AAV9-derived landscape.

4 Summary of files touched today

- **Modified:** `Modelization_V1/lib/sequence_classesV1.py` – NGS depletion (`_ngs.and.deplete`) and Poisson→Binomial fix in `sample_sequences`.
- **Modified:** `Modelization_V1/notebooks/selectivity_weight_regimes/MLP_for_anticorrelated_weights.ipynb` – added F/J decomposition, variance-share, and z-score dominance cells.

- **Added:** `Modelization_V1/notebooks/selectivity_weight_regimes/MLP_bilinear_head_anticorrelated.ipynb` – baseline ProfileMLP vs. ProfileMLP+ bilinear-head comparison.
- **Added:** this report.

5 Open items for a future session

1. Re-run the “more data” test properly controlled (scale D with diversity so per-variant read depth is held constant), to cleanly separate sample size from label noise.
2. Bootstrap `extract_effective_J` to directly measure its own estimation noise floor.
3. Compare J -dominance of the extreme tail on the AAV9-recovered landscape against the synthetic `initialize_random_weights` landscape, where J ’s scale is exact and deliberately small.
4. If minimum-norm shrinkage is confirmed as the driver, consider a low-rank or otherwise shared-strength reparameterization of the bilinear head (fewer effective degrees of freedom than 8,400 independent entries) rather than more data alone.
5. Longer-term: this is precisely the kind of bias that an actual directed-evolution loop (propose candidates $\rightarrow$ get them re-measured $\rightarrow$ retrain) is designed to correct, rather than a one-shot fit from round-1 random data.